

1.6 Notation and conventions

Most notation in the book is introduced as we need it, and we include here only the important asymptotic notation. We also list some conventions and tips.

- **Undefined notions** When a notion unfamiliar to you is mentioned but not immediately defined, then it is not crucial to understand its meaning for comprehending the text (but feel free to look it up e.g. on Wikipedia).
- **References** When a list of references is given, it will typically be chronological. This may not always be evident from the publication year, due mainly to the fact that I was trying always to supply the most comprehensive and up to date version. In some cases these are journal papers (which take longest to appear), some are conference publications (which appear faster), and in rare cases only electronic versions exist (which are instantaneous). Also, when I call a result “recent”, please note that this is relative to the publication date of the book.
- **Footnotes** There are *many* footnotes in this book. In almost all cases, they are designed to enrich, elaborate or further explain something. However, the text itself should be self-contained without them. Therefore you can safely skip footnotes; this will rarely affect understanding.
- **iff** The shorthand *iff* will be used throughout to mean “if and only if”.
- **Asymptotic notation** Crucial to most chapters is the following asymptotic notation, relating “growth in the limit” of functions on the integers. Let f, g be two integer functions. Then we denote:
 - * $f = O(g)$, if for some positive constant C , for all large enough n , $f(n) \leq C \cdot g(n)$.
 - * $f = \Omega(g)$, if for some positive constant c , for all large enough n , $f(n) \geq c \cdot g(n)$.
 - * $f = \Theta(g)$, if both $f = O(g)$ and $f = \Omega(g)$ hold.
 - * $f = o(g)$, if $f(n)/g(n)$ tends to zero as n tends to infinity.

We say that an integer function f grows (at most) *polynomially* if there are constants A, c such that for all n , $f(n) \leq An^c$.

We say that f grows (at most) *exponentially* if there are constants A, c such that for all n , $f(n) \leq A \exp(n^c)$.

2 *Prelude: Computation, undecidability, and limits to mathematical knowledge*

This short section will briefly recount the history of ideas leading to the birth of computational complexity theory.

Mathematical classification problems Which mathematical structures can we hope to understand? Consider any particular class of mathematical objects and any particular relevant property. We seek to *understand* which of the objects have the property and which do not. Examples of this very general *classification problem* include the following¹:

1. Which Diophantine equations have solutions?
2. Which knots are unknotted? (see Figure 1)
3. Which planar maps are 4-colorable? (see Figure 2)
4. Which theorems are provable in Peano arithmetic?
5. Which pairs of smooth manifolds are diffeomorphic?
6. Which elementary statements about the reals are true?
7. Which elliptic curves are modular?
8. Which dynamical systems are chaotic?

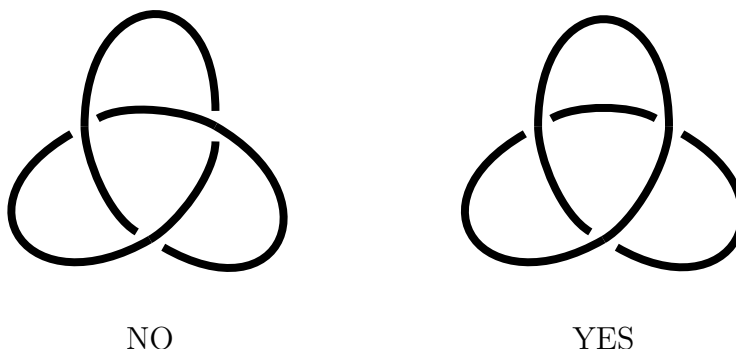


Figure 1. Instances of problem 2 and their classification. The left is a diagram of the Trefoil knot and the right is one of the Unknot.

Understanding and algorithms A central question is what we mean by *understanding*. When are we satisfied that our classification problem has been reasonably solved? Are there problems like these that we can never solve? A central observation (popularized mainly by David Hilbert) is that “satisfactory” solutions usually provide (explicitly or implicitly) mechanical procedures, which when applied to an object, determine (in finite time) whether it has the property. Hilbert’s problems 1 and 4 above were stated, it seems, with the expectation that the answer would be positive; namely, that mathematicians would be able to understand them in this very sense.

¹It is not essential that you understand every mathematical notion mentioned below. If curious, reading a Wikipedia-level page should be more than enough for our purposes here.

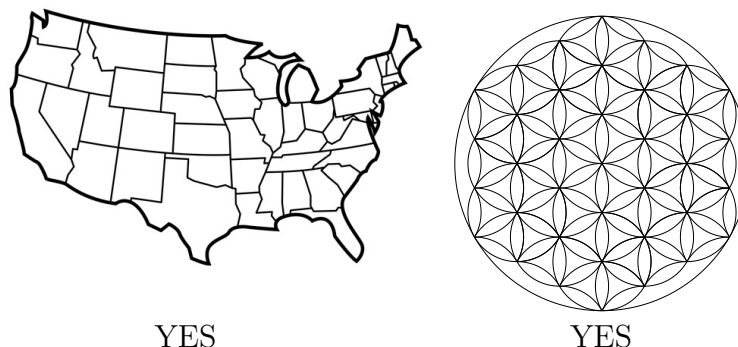


Figure 2. Instances of problem 3 and their classification. Both maps are 4-colorable.

So, Hilbert identified mathematical knowledge with computational access to answers, but never formally defined computation. This task was taken up by logicians in the early twentieth century and was met with resounding success in the 1930s. The breakthrough developments by Gödel, Turing, Church, and others led to several quite different formal definitions of computation, which happily turned out to be identical in power. Of all, Turing’s 1936 paper [Tur36] was most influential. *Indeed, it is easily the most influential math paper in history.* We already mentioned in Section 1.1 that in this extremely readable paper, Turing gave birth to the discipline of computer science and ignited the computer revolution, which radically transformed society. Turing’s model of computation (quickly named a *Turing machine*) became one of the greatest intellectual inventions ever. Its elegant and simple design on one hand, and its universal power (elucidated and exemplified by Turing) on the other immediately led to implementations, and the rapid progress since then has forever changed life on Earth. This paper serves as one of the most powerful demonstrations of how excellent theory predates and enables remarkable technological and scientific advances. But on top of all these, Turing’s paper also resolved Hilbert’s problems 1 and 4! In the negative!! Let us see how.

With the Turing machine, we finally have a rigorous definition of computation, giving formal meaning to Hilbert’s questions. It allows proving mathematical theorems about what “mechanical procedures” can and cannot do! *Turing defined an **algorithm** (also called a decision procedure) to be a Turing machine (in modern language, simply a computer program) that halts on every input in finite time.* So, algorithms compute functions, and being finite objects themselves, one immediately sees from a Cantor-like diagonalization argument that some (indeed almost all) functions are *not* computable by algorithms. Such functions are also called *undecidable*; they have no decision procedure. But Turing went further and showed that specific, natural functions, like Hilbert’s Entscheidungsproblem (4) above was undecidable (this was independently proved by Church as well). Turing’s elegant 1-page proof adapts a Gödelian self-reference argument on a Turing machine.² These demonstrate powerfully the mathematical value of Turing’s basic computational model.

Decidability and undecidability Turing thus shattered Hilbert’s first dream. Problem 4 being undecidable means that we will never understand in general, in Hilbert’s sense, which theorems are provable (say, in Peano arithmetic): No algorithm can discern provable from unprovable theorems. It took 35 more years to do the same to Hilbert’s problem 1. Its undecidability (proved by Davis, Putnam, Robinson, and Matijasevich in 1970) says that we will never understand in this way polynomial equations over integers: No algorithm can discern solvable from unsolvable ones.

A crucial ingredient in those (and all other undecidability) results is showing that each of these mathematical structures (Peano proofs, integer polynomials) can “*encode computation*” (in particular, these seemingly

²As a side bonus, a similar argument gives a short proof of Gödel’s incompleteness theorem, a fact which for some reason is still hidden from many undergraduates taking logic courses.

static objects encode a dynamic process). This is known today to hold for many different mathematical structures in algebra, topology, geometry, analysis, logic, and more, even though *a priori* the structures studied seem to be completely unrelated to computation. This ubiquity makes every mathematician a potential computer scientist in disguise. We shall return to refined versions of this idea later.

Naturally, such negative results did not stop mathematical work on these structures and properties—they merely suggested the study of interesting subclasses of the given objects. Specific classes of Diophantine equations were understood much better, for example, Fermat’s Last Theorem and the resolution of problem (7) regarding the modularity of elliptic curves. The same holds for restricted logics for number theory (e.g. Presburger arithmetic).

The notion of a decision procedure (or algorithm) as a minimal requirement for understanding a mathematical problem has also led to direct positive results. It suggests that we should look for a decision procedure as a *means*, or as the *first step* for understanding a problem. With this goal in mind, Haken [Hak61] showed how knots can be understood in this sense, designing a decision procedure for problem (2), determining knottedness. Similarly Tarski [Tar51] showed that closed real fields can be thus understood, designing a decision procedure for problem (6). Naturally, significant *mathematical*, *structural* understanding was needed to develop these algorithms. Haken developed the theory of *normal surfaces* and Tarski invented *quantifier elimination* for their algorithms; in both cases, these ideas and techniques became cornerstones of their respective fields.

These important examples, and many others like them, only underscore what has been obvious for centuries: mathematical and algorithmic understanding are strongly related and often go hand in hand, as discussed at length in the introduction. And what was true in previous centuries is still true in this one: The language of algorithms is compatible with and actually generalizes the language of equations and formulas (which are special cases of algorithms), and is a powerful language for understanding and explaining complex mathematical structures.

The many decision procedures developed for basic mathematical classification problems, such as Haken’s and Tarski’s solutions to problems (2) and (6), respectively, demonstrate this notion of algorithmic understanding *in principle*. After all, what they guarantee is an algorithm that will deliver the correct solution in *finite* time. Should this satisfy us? Finite time can be very long, and it is hard to distinguish a billion years from infinity. This is not just an abstract question for the working mathematician, as we recounted in Section 1.1. Indeed, both Haken’s and Tarski’s original algorithms were extremely slow, and computing their answer for objects of moderate size may indeed have required a billion years. Can this be quantified?

Efficient algorithms and computational complexity This viewpoint suggests developing and using a computational yardstick and measuring the quality of understanding by the quality of the algorithms providing it. Indeed, we argue that better mathematical understanding of given mathematical structures often goes hand in hand with better algorithms for classifying their properties. Formalizing the notions of algorithms’ resources (especially *time*) and their efficient use is the business of computational complexity theory, the subject of this book, which we shall start developing in the next section. But before we do, I would like to use the set of problems above to highlight a few other issues, which we will not discuss further here.

First, the *representation* of objects which algorithms process becomes important! One basic issue raised by most of the problems above is the contrast between continuity in mathematics and discreteness of computation. Algorithms manipulate finite objects (like bits) in discrete time steps. Knots, manifolds, dynamical systems, and the like are continuous objects. How can they be described to an algorithm and processed by it? As many readers will know, the answers vary, but finite descriptions exist for all. For example, we use knot diagrams for knots, triangulations for manifolds, and symbolic descriptions or successive approximations for dynamical systems. It is these *discrete* representations that are indeed used in algorithms for these (and other) *continuous* problems (as, e.g., Haken’s algorithm demonstrates). Observe that this has to be the case; every continuous object we humans will ever consider has discrete representations! After all, math textbooks and papers are finite sequences of characters from a finite alphabet, just like the input to Turing machines. And we, their readers, would never be able to process and discuss them otherwise. All

this does not belittle the difficulties that may arise when seeking representations of continuous mathematical structures that would be useful for description, processing, and discussion—instead, it further illustrates the inevitable ties between mathematics and computation.

Let me demonstrate that algorithmic efficiency may crucially depend on representation even for simple *discrete* structures. Where would mathematics (and society) be if we continued using *unary* encodings of integers, or even Roman numerals? The *great* invention of the decimal encoding (or more generally, the *positional* number system) was motivated by, and came equipped with, efficient algorithms for arithmetic manipulation! And this is just one extremely basic example.

Problem 3 on the 4-colorability of planar maps points to a different aspect of the interaction of computation and mathematics. Many readers will know that problem (3) has a very simple decision procedure: Answer “yes” on every input. The correctness of this trivial algorithm is guaranteed by the highly nontrivial proof of the 4-color theorem of Appel and Haken [AH89]. This theorem states that in every planar map (as the ones used in geography texts, and Figure 2), each region can be colored from a set of 4 colors (e.g., Red, Blue, Green, Yellow), so that no two regions sharing a border are assigned the same color. This mathematical proof was the first to use a computer program as an essential tool to check that an enormously large but finite number of finite graphs is indeed 4-colorable. The proof naturally raised numerous discussions and arguments on the value of such proofs; discussions which only intensified with time, as more and more proofs use computers in a similar way (another famous example is Hales’s proof of the Kepler conjecture [Hal05]). I leave it to the reader to contemplate the relative merits of computer vs. human generated proofs (and the task of distinguishing the two). Another point to make is that problem (3) may seem to some very different from the rest; all are well-known “deep” problems of mathematics, whereas (3) seems recreational. Indeed, in the twentieth century, it was quite popular in mathematics to call such problems “trivial” even without knowing the 4-color theorem, simply because a trivial brute-force algorithm that tries all (finitely many) possible colorings could determine the answer. This again brings us right back to the quality of algorithms, discussed next. You may want to revisit the task of distinguishing “deep” and “trivial” problems after reading Section 3.1.

3 Computational complexity 101: The basics, $\mathcal{P}$, and $\mathcal{NP}$

In this chapter, we develop the basic notions of computational problems, data representation; efficient computations; efficient reductions between problems; efficient verification of proofs; the classes $\mathcal{P}$, $\mathcal{NP}$, and $\text{co}\mathcal{NP}$; and the notion of $\mathcal{NP}$ -complete problems. We focus on *classification* (or decision) problems; other types of problems, and classes, (enumeration, approximation, construction, etc.) are studied as well, and some will be discussed later in the book.

When studying efficiency, we will focus on *time* as the primary resource of algorithms. By “time”, I mean the number of elementary operations¹ performed. Other resources of algorithms, such as memory, parallelism, communication, and randomness, are studied in computational complexity, and some will be treated later in the book.

3.1 Motivating examples

Let us consider the following three classification problems. As in Chapter 2, for each classification (or decision) problem like these, we get a description of an object and have to decide whether it has the desired property.

(1'.) Which Diophantine equations of the form $Ax^2 + By + C = 0$ are solvable by positive integers?

(2'.) Which knots on 3-dimensional manifolds bound a surface of genus $\leq g$?

(3'.) Which planar maps are 3-colorable?

Problem (1') is a restriction of problem (1) in Chapter 2. Problem (1) was undecidable, and it is natural to try to better understand more restricted classes of Diophantine equations. Problem (2') is a generalization of problem (2) in Chapter 2 in two ways: The unknotting problem (2) considers the special case of the manifold $\mathbb{R}^3$ and genus $g = 0$. Problem (2) was decidable, and so we may want to know whether its generalization (2') is decidable as well. Problem (3') is an interesting variant of problem (3). While every map has a 4-coloring, not every map has a 3-coloring; some do and some don't, and so this is another nontrivial classification problem to understand.

Most mathematicians would tend to agree that these three problems have absolutely nothing to do with one another. They are each from very different fields—algebra, topology, and combinatorics, respectively—each with its completely different notions, goals, and tools. However, the theorem below suggests that this view may be wrong.

Theorem 3.1. *Problems (1') , (2') and (3') are equivalent.*

Moreover, the equivalence notion is natural and completely formal. Intuitively, any understanding we have of one problem can be *simply* translated into a similar understanding of the other. The formal meaning of this equivalence will unfold in this chapter and be formalized in Section 3.9. To get there, we need to develop the language and machinery that yield such surprising results.

Representation issues We start by discussing (informally and by example) how such varied complex mathematical objects can be described in finite terms, eventually as a sequence of bits. Often there are several alternative representations, and typically it is simple to convert one to the other. Let us discuss the representation of inputs in these three problems.

For problem (1') consider first the set of all equations of the form $Ax^2 + By + C = 0$ with integer coefficients A, B, C . A finite representation of such equations is obvious—the triple of coefficients (A, B, C) , say with each integer written in binary notation. Given such a triple, the decision problem is whether the corresponding polynomial has a positive integer root (x, y) . Let $2DIO$ denote the subset of triples for which the answer is YES.

¹For example, reading/writing a finite amount of data from/to memory, or performing a logical or arithmetic operation involving a finite amount of data.

Finite representation of inputs to problem (2') is tricky but still natural. The inputs consist of a 3-dimensional manifold M , a knot K embedded in it, and an integer G . A finite representation can describe M by a triangulation (a finite collection of tetrahedra and their adjacencies). The knot K will be described as a closed path along edges of the given tetrahedra. Given a triple (M, K, G) , the decision problem is whether a surface that K bounds has genus at most G . Let $KNOT$ denote the subset for which the answer is YES.

Finite representation of inputs to problem (3') is nontrivial as well. Let us discuss not maps but instead graphs, in which vertices represent the countries and edges represent adjacency of countries (this view is equivalent; for a planar map, its graph is simply its dual map). To describe a graph (in a way that makes its planarity evident), one elegant possibility is to use a simple and beautiful theorem of Fáry [Fár48] (discovered independently by others and has many proofs). It states that every planar graph has a *straight line* embedding in the plane (with no edges crossing). So, the input can be a set V of coordinates of the vertices (which can in fact be small integers) and a set E of edges, each a pair of elements from V . Let $3COL$ be the subset of those inputs (V, E) describing a 3-colorable map.

Any finite object (integers, tuples of integers, finite graphs, finite complexes, etc.) can be represented naturally by binary sequences over the alphabet $\{0, 1\}$, and this is how they will be described as inputs to algorithms. As discussed above, even continuous objects like knots have finite descriptions and so can be described this way as well.² We will not discuss here subtle issues like whether objects have unique representations or whether every binary sequence should represent a legal object. It suffices to say that in most natural problems, this encoding of inputs can be chosen such that these are not real issues. Moreover, going back and forth between the object and its binary representation is simple and efficient (a notion to be formally defined below).

Consequently, let $\mathbf{I}$ denote the set of all finite binary sequences, and regard it as the set of inputs to all our classification problems. Indeed, every subset of $\mathbf{I}$ defines a classification problem. In this language, given a binary sequence $x \in \mathbf{I}$, we may interpret it as a triple of integers (A, B, C) and ask whether the related equation is in $2DIO$. This is problem (1'). We can also interpret x as a triple (M, K, G) of manifold, knot, and integer, and ask whether it is in the set $KNOT$. This is problem (2'), and the same can be done with (3').

Reductions Theorem 3.1 states that there are *simple* translations (in both directions) between solving problem (1') and problem (2'). More precisely, it provides efficiently computable functions $f, h: \mathbf{I} \rightarrow \mathbf{I}$ performing these translations:

$$(A, B, C) \in 2DIO \text{ iff } f(A, B, C) \in KNOT,$$

and

$$(M, K, G) \in KNOT \text{ iff } h(M, K, G) \in 2DIO.$$

Thus, an efficient algorithm to solve one of these problems immediately implies a similar one for the other. Putting it more dramatically, if we have gained enough understanding of topology to solve, say, the knot genus problem, it means that we automatically have gained enough number theoretic understanding for solving these quadratic Diophantine problems (and vice versa).

The translating functions f and h are called *reductions*. We capture the *simplicity* of a reduction in *computational* terms, demanding that it will be *efficiently* computable.

Similar pairs of reductions exist between the map 3-coloring problem and each of the other two problems. If sufficient understanding of graph theory leads to an efficient algorithm to determine whether a given planar map is 3-colorable, similar algorithms follow for both $KNOT$ and $2DIO$. And vice versa—understanding either of them will similarly resolve 3-coloring. Note that this positive interpretation of the equivalence paints all three problems as equally “accessible.” But the flip side says that they are also equally intractable: If any one of them lacks such an efficient classification algorithm, so do the other two. Indeed, with the better

²Theories of algorithms which have continuous inputs (e.g., real or complex numbers) have been developed, for example, in [BCSS98, BC06], but will not be discussed here.

understanding of these equivalences today, it seems more likely that the second interpretation is right: These problems are all hard to understand.

When teaching this material in class, or in lectures to unsuspecting audiences, it is always fun to watch listeners' amazement at these remarkably strong and unexpected connections between such remote problems. I hope it has a similar impact on you. But now it is time to dispel the mystery and explain the source of these connections. Here we go.

3.2 Efficient computation and the class $\mathcal{P}$

Efficient algorithms are the engine which drive an ever growing part of industry and economy, and with it your everyday life. These jewels are embedded in most devices and applications you use daily. In this section we abstract a mathematical notion of efficient computation, polynomial-time algorithms. We motivate it and give examples of such algorithms.

In all that follows, we focus on asymptotic complexity. Thus e.g. we care neither about the time it takes to factor the number $2^{67} - 1$ (as much as Mersenne cared about it), nor about the time it takes to factor all 67-bit numbers, but rather about the asymptotic behavior of factoring n -bit numbers, as a function of the input length n . The asymptotic viewpoint is inherent to computational complexity theory, and we shall see in this book that it reveals structure which would be obscured by finite, precise analysis. We note that the dependence on input size does not exist in Computability theory, where algorithms are simply required to halt in finite time. However, much of the methodology of these fields was imported to computational complexity theory—complexity classes of problems, reductions between problems and complete problems, all of which we shall meet.

Efficient computation (for a given problem) will be taken to be one whose runtime on any input of length n is bounded by a *polynomial* function in n .

Recall that $\mathbf{I}$ denotes the set of all binary sequences of all lengths. Let $\mathbf{I}_n$ denote all binary sequences in $\mathbf{I}$ of length n , namely $\mathbf{I}_n = \{0, 1\}^n$.

Definition 3.2 (The class $\mathcal{P}$). A function $f: \mathbf{I} \rightarrow \mathbf{I}$ is in the class $\mathcal{P}$ if there is an algorithm computing f and positive constants A, c , such that for every n and every $x \in \mathbf{I}_n$, the algorithm computes $f(x)$ in at most An^c steps (namely, elementary operations).

Note that the definition applies in particular to Boolean functions (whose output is $\{0, 1\}$), which capture classification problems (often called “decision problems”). We will abuse notation and sometimes think of $\mathcal{P}$ as the class containing *only* these classification problems. Observe that a function with a long output can be viewed as a sequence of Boolean functions, one for each output bit.

This important definition, contrasting polynomial growth with (brute force) exponential growth, was put forth in the late 1960s by Cobham [Cob65], Edmonds [Edm65b, Edm67a], and Rabin [Rab67]. These researchers, coming from different areas and motivations, all attempted to formally delineate *efficient* from just *finite* algorithms. Of course, nontrivial polynomial-time algorithms were discovered earlier, long before the computer age. Many were discovered by mathematicians who needed efficient methods to calculate (by hand). The most ancient and famous example is of course Euclid’s GCD (greatest common divisor) algorithm mentioned in Chapter 1, which was invented to bypass the need to factor integers when computing their common factor.

Two major choices must be made in selecting $\mathcal{P}$ to model the class of efficiently computable functions, which are often debated and certainly demand explanation. One is the choice of *polynomial* as the bound on time in terms of input length, and the second is the choice of *worst-case* requirement (namely, that this time bound holds for all inputs). We discuss the motivation and importance of these two choices below. However, it is important to stress that these choice are not dogmatic: Computational complexity has considered and investigated numerous other alternatives to these choices (e.g., many finer grained bounds on efficiency other than polynomial, and many different notions of average case and input-dependent measures replacing the worst-case demands). Some of them will be discussed later in the book. Still, these choices were extremely important in the early days of computational complexity, revealing beautiful structure that would become the solid foundation for the field and enable the subsequent study of finer and more diverse alternatives.

3.2.1 Why polynomial?

The choice of polynomial time to represent efficient computation seems arbitrary. However, this particular choice has justified itself over time from many points of view. I list some important justifications.

Polynomials typify “slowly growing” functions. The closure of polynomials under addition, multiplication, and composition preserves the notion of efficiency under natural programming practices, such as using two programs in sequence, or using one as a subroutine of another. This choice removes the necessity of describing the computational model precisely (e.g., it does not matter whether we allow arithmetic operations only on single digits or on arbitrary integers, since long addition, subtraction, multiplication, and division have simple polynomial-time algorithms taught in grade school). Similarly, we need not worry about data representation: one can efficiently translate between essentially any two natural representations of a set of finite objects.

From a practical viewpoint, a running time of, say, n^2 is far more desirable than n^{100} , and of course linear time is even better. Indeed even the constant coefficient of the polynomial running time can be crucial for real-life feasibility of an algorithm. However, there seems to be a “law of small numbers” at work: Very few known polynomial-time algorithms for natural problems have exponents above 3 or 4 (even though at discovery, the initial exponent may have been 30 or 40). However, many important natural problems that so far resist any efficient algorithms cannot at present be solved faster than in *exponential* time (which of course is totally impractical, even for small input data). This exponential gap gives great motivation for the definition of $\mathcal{P}$; reducing the complexity of such problems from exponential to (any) polynomial will be a huge conceptual improvement, likely involving new techniques.

3.2.2 Why worst case?

Another criticism of the definition of the class $\mathcal{P}$ is that a problem is deemed efficiently solvable if *every* input of length n can be solved in $\text{poly}(n)$ -time. From a practical standpoint, it would suffice that the instances we care about (e.g., those generated by our application, be it in industry or nature) be solved quickly by our algorithms, and the rest can take a long time. Perhaps it suffices that “typical” instances be solved quickly. Of course, understanding what instances arise in practice is a great problem in itself, and a variety of models of typical behavior and algorithms for them are studied (and we shall touch upon this in section 4.4). The clear advantage of worst-case analysis is that we don’t have to worry about which instances arise—they will all be solved quickly by what we call an “efficient algorithm”. This notion “composes” well (i.e., when one algorithm is using another). Moreover, it accounts for adversarial situations where an input (or more generally, external behavior) is generated by an unknown opponent, who wishes to slow down the algorithm (or system). Modeling such adversaries is crucial in such fields as cryptography and error correction, and it is facilitated by worst-case analysis. Finally, as mentioned, this notion turned out to reveal a very elegant structure of the complexity universe, which inspired the more refined study of average-case and instance-specific theories.

Understanding the class $\mathcal{P}$ is central. Numerous computational problems arise (in theory and practice) that demand efficient solutions. Many algorithmic techniques were developed in the past four decades and enable solving many of these problems (see, e.g., the textbooks [CLR01, KT06]). These techniques drive the ultrafast home computer applications we now take for granted, like web searching, spell checking, data processing, computer game graphics, and fast arithmetic, as well as heavier duty programs used across industry, business, math, and science. But many more problems (some of which we shall meet soon), perhaps of higher practical and theoretical value, remain elusive. The challenge of *characterizing* this fundamental mathematical object—the class $\mathcal{P}$ of efficiently solvable problems—is far beyond us at this point.

We end this section with some examples of nontrivial problems in $\mathcal{P}$ of mathematical significance from diverse areas. In each, the interplay between mathematical and computational understanding needed for the development of these algorithms is evident. Most examples are elementary in nature, but if some mathematical notion is unfamiliar, feel free to ignore that example (or possibly better, look up its meaning).

3.2.3 Some problems in $\mathcal{P}$

- **Perfect matching.** Given a graph, test whether it has a *perfect matching*, namely, a pairing of all

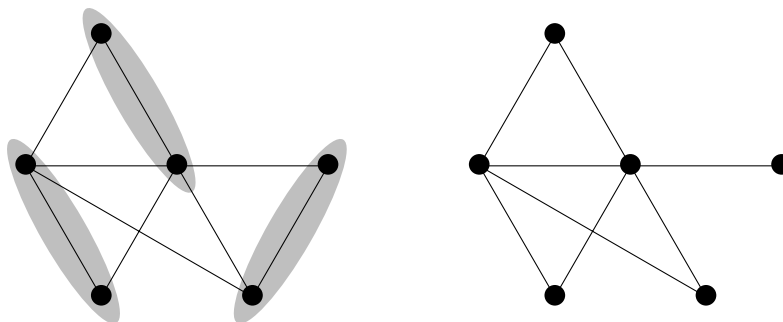


Figure 3. A graph with a perfect matching (left; matching is shown) and one without a perfect matching.

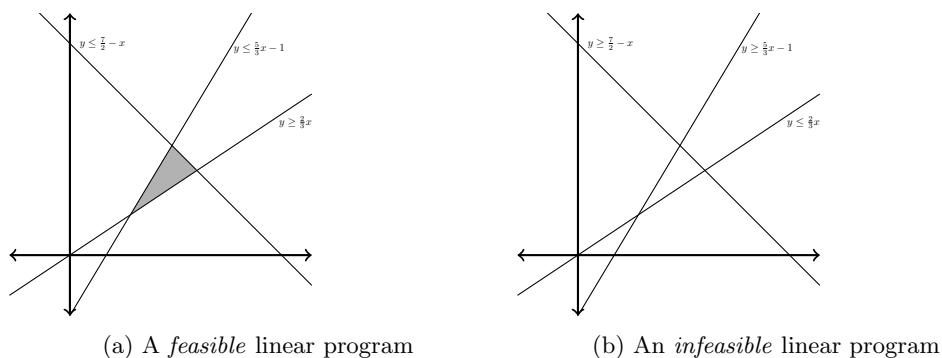


Figure 4. Two linear programs with two variables and three inequalities.

its vertices such that every pair is an edge of the graph (see Figure 3). The ingenious algorithm of Edmonds [Edm65b] is probably the first nontrivial algorithm in $\mathcal{P}$, and as mentioned above, this paper was central to highlighting $\mathcal{P}$ as an important class to study. The structure of matchings in graphs is one of the most well-studied subject in combinatorics (see, e.g., [LP09]).

- **Primality testing.** Given an integer, determine whether it is prime.³ Gauss literally appealed to the mathematical community to find an efficient algorithm, but it took two centuries to resolve. The story of this recent achievement of Agrawal, Kayal, and Saxena [AKS04] and its history is beautifully recounted by Granville in [Gra05]. Of course, there is no need to elaborate on how central prime numbers are in mathematics (and even in popular culture).
- **Planarity testing.** Given a graph, determine whether it is *planar*. Namely, can it be embedded in the plane without any edges crossing? (Try to determine this for the graphs in Figure 3 and those in Figure ??.) A sequence of *linear* time algorithms for this basic problem was discovered, starting with the paper of Hopcroft and Tarjan [HT74].
- **Linear programming.** Given a set of linear inequalities in many variables, determine whether they are mutually consistent, namely, whether there are real values for the variables satisfying all inequalities (a simple example is shown in Figure 4). This problem, and its optimization version, is enormously useful in applications. It captures many other problems (e.g., finding optimal strategies of zero-sum games). The convex optimization techniques used to give the efficient algorithms ([Kha79], [Kar84]) for it actually do much more (see, e.g., Schrijver’s book [Sch03]).

³For example, try to determine the answer for $X - 1$ and $X + 1$, where $X = 6797727 \times 2^{15328}$.

- **Factoring polynomials.** Given a multivariate polynomial with rational coefficients, find its irreducible factors over $\mathbb{Q}$. The tools developed by Lenstra, Lenstra, and Lovász in [LLL82] (mainly regarding short bases in lattices in $\mathbb{R}^n$) have many other applications (see Section 13.8).
- **Hereditary graph properties.** Given a finite graph, test whether it is a member of some fixed minor-closed family.⁴ A polynomial-time algorithm (which has a huge exponent in general) follows Robertson and Seymour’s monumental structure theory [RS95] of such families, including a finite basis theorem.⁵
- **Permutation group membership.** Given a list of permutations on n elements, can the first one be generated by the rest?⁶ The non-commutative Gaussian elimination techniques developed in [Sim70, FHL80] triggered development of algorithmic group theory, which is used extensively by group theorists and in particular led to the breakthrough in testing graph isomorphism [Bab15].⁷
- **Hyperbolic word problem.** Given any presentation of a *hyperbolic* group by generators and relations, and a word w in the generators, check whether w represents the identity element. Gromov’s geometric techniques, including isoperimetric bounds on the Cayley graphs of such groups [Gro87], allow a polynomial-time algorithm (and more). For general finitely presented groups, this problem is undecidable.

3.3 Efficient verification and the class $\mathcal{NP}$

Let $C \subset \mathbf{I}$ be a classification problem.⁸ Specifically, we are given as input a binary sequence $x \in \mathbf{I}$ (describing a mathematical object), and are supposed to determine whether $x \in C$. It is convenient to view C as defining a *property*; $x \in C$ are objects having the property, and $x \notin C$ are objects that do not. If we have an efficient algorithm for C , we can simply apply it to x and know whether x has property C . But if we do not have such an algorithm, what is the next best thing? One answer is, a *convincing proof* that $x \in C$. Before defining it formally, let us see a couple of motivating examples.

The first example is a famous anecdote of a lecture given by F. N. Cole, titled “On the Factorization of Large Numbers,” given at the 1903 American Math Society meeting. Without uttering a word, he went to the blackboard, wrote

$$2^{67} - 1 = 147573952589676412927 = 193707721 \times 761838257287,$$

and proceeded to perform the long multiplication of the integers on the right-hand side to derive the integer on the left: Mersenne’s 67th number (which was conjectured to be prime). No one in the audience had any questions.

What happened there? Cole demonstrated that the number $2^{67} - 1$ is *composite*. Indeed, we can see that such a short proof can be given for any (correct) claim of the form $x \in \text{COMPOSITES}$, with *COMPOSITES* denoting the set of composite numbers. The proof can simply be a factorization of x . The features we want to extract from this episode are two: the proofs here are *short* and *easily verifiable*. Indeed, the total length of the factors is roughly the length of the input, and multiplying them has an efficient algorithm. Note that the fact that it was extremely difficult for Cole to *find* these factors (he said it took him “three years of Sundays”) did not affect in any way that demonstration.

A second example, which many of us meet daily, is what happens when we read a typical math journal paper. In it, we typically find a (claimed) theorem, followed by an (alleged) proof. Thus, we are verifying claims of the type $x \in \text{THEOREMS}$, where *THEOREMS* is the set of all provable statements in, say, set theory. It is taken for granted that the written proof is *short* (page limit) and *easily verifiable* (a referee

⁴Namely, removing a vertex (or edge), and contracting an edge leave a graph in the family.

⁵Every such family has a finite number of excluded minors.

⁶A famous special case is the question, given a color pattern of a Rubik’s cube (perhaps obtained by placing colored stickers illegally), can it be sorted to monochromatic faces by legal Rubik moves?

⁷See also the exposition [HBD17] of this result.

⁸In the computer science literature, C is often called a *language*.

can verify it in a reasonable time), so at least on an intuitive level *THEOREMS* has the same properties as *COMPOSITES*, and this can be made formal. Note again that we don't care how long it took the authors to *find* the proof. Needless to say, theorems and proofs in mathematical journals are not really written in a formal language; indeed, one can interpret the task of refereeing as verifying that these “semi-formal” proofs could be converted into formal ones that will establish the truth of the statements they claim to prove.

Now we are ready for a definition of $\mathcal{NP}$, the class of problems generalizing these two examples.

The class $\mathcal{NP}$ contains all properties C for which membership (namely, statements of the form $x \in C$) have *short, efficiently verifiable* proofs. As before, we use polynomials to define both terms. A candidate proof y for the claim $x \in C$ must have length at most polynomial in the length of x . And the verification that a given y indeed proves the claim $x \in C$ must be checkable in polynomial time (via a verification algorithm we will call V_C). Finally, if $x \notin C$, no such y should exist. Let us formalize this definition.

Definition 3.3 (The class $\mathcal{NP}$). The set C is in the class $\mathcal{NP}$ if there is a function $V_C \in \mathcal{P}$ and a constant k such that

- If $x \in C$, then $\exists y$ with $|y| \leq k \cdot |x|^k$ and $V_C(x, y) = 1$;
- If $x \notin C$, then $\forall y$ we have $V_C(x, y) = 0$.

From a logic standpoint, each set C in $\mathcal{NP}$ may be viewed as a set of theorems in the complete and sound proof system defined by the *verification process* V_C .

A sequence y that “convinces” V_C that $x \in C$ is often called a *witness* or *certificate* for the membership of x in C . Again, we stress that the definition of $\mathcal{NP}$ is not concerned with how difficult it is to come up with a witness y , but rather only with the efficient verification using y that $x \in C$. The witness y (if it exists) can be viewed as given by an omnipotent entity, or simply guessed. Indeed, the acronym $\mathcal{NP}$ stands for “Nondeterministic Polynomial time,” where the nondeterminism captures the ability of a *hypothetical* nondeterministic machine to “guess” a witness y (if one exists) and then verify it deterministically.

Nonetheless, the complexity of finding a witness is, of course, important, as it captures the *search problem* associated with $\mathcal{NP}$ sets. Every decision problem C (indeed, every verifier V_C for C) in $\mathcal{NP}$ defines a natural search problem associated with it: Given $x \in C$, *find* a short witness y that “convinces” V_C of this fact. A correct solution to this search problem can be efficiently verified by V_C , by definition.

It is clear that finding a witness (if one exists) can be done by brute-force search: as witnesses are short (of length $\text{poly}(n)$ for a length- n input), one can enumerate all possible ones, and to each apply the verification procedure. However, this enumeration takes *exponential time* in n . The major question of this chapter (and this book, and the theory of computation!) is whether much faster algorithms than brute-force exist for *all* $\mathcal{NP}$ problems.

While it is usually the search problems that arise more naturally, it is often more convenient to study the decision versions of these problems (namely, whether a short witness exists). In almost all cases, both decision and search versions are computationally equivalent.⁹

Here is a list of some problems (or rather properties) in $\mathcal{NP}$. Note that some are variants of the problems in the similar list we gave for the class $\mathcal{P}$. However, we have no idea whether any of these are in $\mathcal{P}$. It is a good exercise (easy for most but not all examples) for the reader to define for each of them the short, easily verifiable witnesses for inputs having the property.¹⁰

Some problems in $\mathcal{NP}$:

- **Hamiltonian cycle in graphs.** The set of graphs having a Hamilton cycle, namely, a cycle of edges passing through every vertex exactly once (see Figure 5).
- **Factoring integers.** Triples of integers (x, a, b) , such that x has a prime factor in the interval $[a, b]$.

⁹A notable possible exception is the set *COMPOSITES* and the suggested verification procedure for it, accepting as witness a nontrivial factor. Note that while *COMPOSITES* $\in \mathcal{P}$ is a decision problem, the related search problem is equivalent to integer factorization, which is not known to have an efficient algorithm.

¹⁰The one difficult exception is Matrix Group Membership, which, if you cannot resolve yourself, peek in the beautiful [BS84].

- **Integer programming.** Sets of linear inequalities in many variables that have an integer solution.
- **Matrix group membership.** Triples (A, B, C) of invertible matrices (say, over $\mathbb{F}_2$) of the same size, such that A is in the subgroup generated by B, C .
- **Graph isomorphism.** Pairs of graphs that are isomorphic, namely, having a bijection between their vertex sets that extends to a bijection on their edge sets. (Find which pairs of graphs in Figure 5 are isomorphic.)
- **Polynomial root.** Multivariate polynomials of degree 3 over $\mathbb{F}_2$ that have a root (namely, an assignment to the variables on which it evaluates to 0).

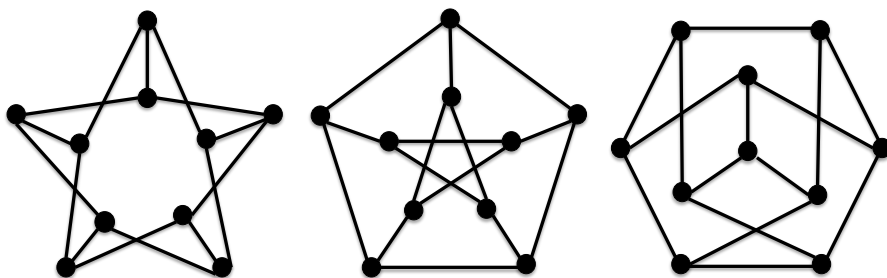


Figure 5. Which of these graphs are Hamiltonian? Which pairs of these graphs are isomorphic?

It is evident that decision problems in $\mathcal{P}$ are also in $\mathcal{NP}$. The verifier V_C is simply taken to be the efficient algorithm for C , and the witness y can be the empty sequence.

Corollary 3.4. $\mathcal{P} \subseteq \mathcal{NP}$.

But can we solve all $\mathcal{NP}$ problems efficiently? Can we vastly improve the trivial “brute-force” exponential time algorithm mentioned above to polynomial time for all $\mathcal{NP}$ problems? This is the celebrated $\mathcal{P}$ vs. $\mathcal{NP}$ question.

Open Problem 3.5. Is $\mathcal{P} = \mathcal{NP}$?

The definition of $\mathcal{NP}$, and the explicit $\mathcal{P} = \mathcal{NP}$? question (and much more that we will soon learn about) appeared formally first (independently and in slightly different forms) in the papers of Cook [Coo71] and Levin [Lev73] in the early 1970s, one researcher in America and the other in the Soviet Union. However, both the definition and question appeared informally earlier, again independently in the East and West, but with similar motivations. They all struggle with the *tractability* of solving problems for which *finite* algorithms exist, including finding finite proofs of theorems, short logical circuits for Boolean functions, isomorphism of graphs, and a variety of optimization problems of practical and theoretical interest. In all these examples, exhaustive search was an obvious but exponentially expensive solution, and the goal of improving it by a possibly more clever (and faster) algorithm was sought, hopefully one of polynomial complexity (namely, in $\mathcal{P}$). The key recognition was identifying the superclass $\mathcal{NP}$ that so neatly encompasses almost all the seemingly intractable problems mentioned above that people really cared about and struggled with.

The excellent survey of Sipser [Sip92] describes this history and gives excerpts from important original papers. Here I mention only a few precursors to the papers above. In the Soviet Union, Yablonskii and his school studied *Perebor*, literally meaning “exhaustive, brute-force search,” and Levin’s paper continues this line of research (see Trakhtenbrot’s survey [Tra84] of this work, including a corrected translation of Levin’s paper). In the West, Edmonds [Edm66] was the first to explicitly suggest “good characterization” of the short, efficiently verifiable type (which he motivates by a teacher-student interaction, although in a slightly stricter

sense than $\mathcal{NP}$, which we will soon meet in Section 3.5). But already in 1956, a decade earlier, a remarkable letter (discovered only in the 1990s, see translation in [Sip92]) written by Gödel to von Neumann essentially introduces $\mathcal{P}$, $\mathcal{NP}$, and the $\mathcal{P}$ vs. $\mathcal{NP}$ question in rather modern language. It raises this fundamental problem of overcoming brute-force search, exemplifies that it is sometimes nontrivially possible, and demonstrates clearly how aware Gödel was of the significance. Unfortunately, von Neumann was already dying of cancer at the time, and it is not known whether he ever responded or Gödel had further thoughts on the subject.

Another very appealing feature of the $\mathcal{P}$ vs. $\mathcal{NP}$ question (which was a source of early optimism about its possible quick resolution) is that it can be naturally viewed as a *bounded* analog of the decidability question from computability theory, which we already discussed implicitly in Chapter 2. To see this, replace the *polynomial-time* bound by a *finite* bound in both classes. For $\mathcal{P}$, the analog becomes all problems having finite algorithms, namely the decidable problems, sometimes called *Recursive* problems and denoted by $\mathcal{R}$. For $\mathcal{NP}$, the analog is the class of properties for which membership can be certified by a finite witness via a finite verification algorithm. This class of problems is called *Recursively Enumerable*, or $\mathcal{RE}$. It is easy to see that most problems mentioned in Chapter 1 are in this class. For example, consider the properties defined by problems 1 and 4 from Chapter 2, respectively the solvable Diophantine equations and the theorems provable in Peano arithmetic. In the first, an integer root of a polynomial is clearly a finite witness that can be easily verified by evaluation in finite time. In the second, a Peano proof of a given theorem is a finite witness, and the chain of deductions of the proof can be easily verified in finite time. Thus, both problems are in $\mathcal{RE}$. We already know that both problems are undecidable (namely, are not in $\mathcal{R}$) and so can conclude that $\mathcal{R} \neq \mathcal{RE}$.

With nearly a half century of experience, we realize that resolving $\mathcal{P}$ vs. $\mathcal{NP}$ is much harder than $\mathcal{R}$ vs. $\mathcal{RE}$. A possible analogy (with a much longer history) is the difficulty of resolving the Riemann Hypothesis, though we have known for millennia that there are infinitely many primes. In both contexts, what we already know is very qualitative, separating the finite and infinite, and what we want to know are very precise, quantitative versions. Also for both problems, some weak quantitative results were proved along the way. The prime number theorem is a much finer quantitative result about the distribution of primes than their infinitude. In this book, we will discuss analogous quantitative progress on the computational complexity of natural problems. In both cases, the long-term goals seem to require much deeper understanding of the respective fields and far better tools and techniques. Incidentally, we will discuss a completely different analogy between the $\mathcal{P}$ vs. $\mathcal{NP}$ question and the Riemann hypothesis in Chapter 8.

3.4 The $\mathcal{P}$ vs. $\mathcal{NP}$ question: Its meaning and importance

Should you care about the $\mathcal{P}$ vs. $\mathcal{NP}$ question? The previous sections make a clear case that it is a very important question of computer science. It is also a precise mathematical question. How about its importance for mathematics? For some mathematicians, the presence of this question in the list of seven Clay Millennium Prize Problems [CJW06], alongside the Riemann Hypothesis and the Poincaré conjecture (which has since been resolved), may be sufficient reason to care. After all, these problems were selected by top mathematicians in the year 2000 as major challenges for the next millennium, each carrying a prize of one million dollars for its solution.

In this section, I hope to explain the ways in which the $\mathcal{P} = \mathcal{NP}$ question is unique not only among the Clay problems but also among all mathematics questions ever asked, in its immense practical and scientific importance, and its deep philosophical content. In a (very informal, sensational) nutshell, it can be summarized as follows:

Can we solve all the problems we can “legitimately” hope to solve?

where the royal “we” can stand for anyone or everyone, representing the general human thrust for knowledge and understanding. In particular, this phrasing of the $\mathcal{P} = \mathcal{NP}$ question clearly addresses the possibility of resolving extant and future conjectures and open problems raised by mathematicians (at the very least, problems regarding classifications of mathematical objects).

To support this overarching interpretation of the $\mathcal{P} = \mathcal{NP}$ question a, let us try to understand at a high level and in intuitive terms which problems occupy these two important classes. In fact, we have already

intuitively identified the class $\mathcal{P}$ with a good approximation of all problems we can solve (efficiently, e.g., in our lifetimes). So next we embark on intuitively identifying $\mathcal{NP}$ as a good approximation of all “interesting” problems: those we are really investing effort in trying to solve, believing that we possibly can. Note that any argument for this interpretation will have to explain why undecidable problems (that are clearly not in $\mathcal{P}$) are not really “interesting” in this sense.

The very idea that all (or even most, or even very many) “interesting” problems can be mathematically identified is certainly audacious. Let us consider it, progressing slowly. We caution that this discussion is mainly of a philosophical nature, and the arguments I make here are imprecise and informal, representing my personal views. I encourage the reader to poke holes in these arguments, but I also challenge you to consider whether counterexamples found to general claims made here are typical or exceptional. After this section, we shall soon return to the sure footing of mathematics!

So, which problems occupy $\mathcal{NP}$? The class $\mathcal{NP}$ turns out to be extremely rich. There are literally thousands of $\mathcal{NP}$ problems in mathematics, optimization, artificial intelligence, biology, physics, economics, industry, and more that arise naturally out of very different necessities, and whose efficient solutions will benefit us in numerous ways. *What is common to all these possibly hard problems, which nevertheless separates them from certainly hard problems (like undecidable ones)?*

To explore this, it is worthwhile to consider a related question: *What explains the abundance of so many natural, important, diverse problems in the class $\mathcal{NP}$?* After all, this class was defined as a technical, mathematical notion by computational theorists. Probing the intuitive meaning of the definition of $\mathcal{NP}$, we will see that it captures many tasks of human endeavor *for which a successful completion can be easily recognized*. Consider the following professions, and the typical tasks they are facing (this list will be extremely superficial, but nevertheless instructive):

- **Mathematician:** Given a mathematical claim, come up with a proof for it.
- **Scientist:** Given a collection of data on some phenomena, find a theory explaining it.
- **Engineer:** Given a set of constraints (on cost, physical laws, etc.), come up with a design (of an engine, bridge, laptop, etc.) that meets these constraints.
- **Detective:** Given the crime scene, find “who done it.”

Consider what may be a common feature of this multitude of tasks. I claim that in almost all cases, “we can tell” a good¹¹ solution when we see one (or we at least believe that we can). Simply put, *would you embark on a discovery process if you didn’t expect to recognize what you set out to find?* It would be good for the reader to consider this statement seriously and try to look for counterexamples. Of course, in different settings the “we” above may refer to members of the academic community, customers of various products, or the juries in different trials. I have had many fun discussions, especially after lectures on the subject, of whether scientists or even artists are indeed in the mental state described. It seems to me that in these cases, the very decision to tell (or not) others of our creations typically follows the application of such a “goodness test” to our work. If so, embarking on the task, we (implicitly or explicitly) expected the solution (or creation) to essentially bear the burden of proof of goodness that we can test, namely, be *short* and *efficiently verifiable*, just as in the definition of $\mathcal{NP}$.

The richness of $\mathcal{NP}$ follows from the simple fact that such tasks abound, and their mathematical formulation is indeed an $\mathcal{NP}$ problem. For all these tasks, *finding* solutions quickly is paramount, and so the importance of the $\mathcal{P}$ vs. $\mathcal{NP}$ question is evident. The colossal implications of the possibility that $\mathcal{P} = \mathcal{NP}$ are evident as well: as that $\mathcal{P}$ represents efficiently solvable problems, we conclude that every instance of all these tasks can be solved efficiently.¹² Optimal solutions to humanity’s most burning questions—medical, social, industrial, scientific, mathematical, and so forth—would be generated instantly (this is discussed in

¹¹In this context, “good” may mean “optimal,” or “better than previous ones,” or “publishable,” or any criterion we establish for ourselves.

¹²In fact, a much larger class of problems, which include some without any clear way of recognizing solutions, will also become easy to solve.

great detail and with many examples in Fortnow’s popular book [For13]). A positive answer to one precise mathematical question holds the key to achieving this utopia! I believe that this universal promise seems to distinguish the $\mathcal{P}$ vs. $\mathcal{NP}$ question from every other mathematical question ever asked.

One can cast doubt on the strong statement above on several grounds. First, while most problems considered by humans may be of this nature (namely, their solutions are easily recognizable) using $\mathcal{P} = \mathcal{NP}$ (if true) to find these solutions requires the efficient recognition procedure to be fully specifiable formally. My reaction to this requirement is that for many important problems (especially in math, science, and engineering), such procedures already exist. For other important problems, if $\mathcal{P} = \mathcal{NP}$ is proved, there will be a huge incentive to convert intuitive recognition procedures into formal ones in order to use them. Another doubt may be that the polynomial-time algorithms supplied by $\mathcal{P} = \mathcal{NP}$ may be too slow in practice (e.g., because the polynomial time bound or the constants involved are too large). This indeed is possible and will in turn provide huge incentives to improve the efficiency of the algorithm. As discussed in Section 3.2, most of these problems currently have only brute-force, exponential-time algorithms, and a polynomial-time algorithm (even an inefficient one) must represent significant new understanding, which should now be fine-tuned to become more efficient.

So, should we believe that $\mathcal{P} = \mathcal{NP}$, and that such utopia is achievable? One (psychological) reason people feel that $\mathcal{P} = \mathcal{NP}$ is unlikely is that such tasks as listed above often require a degree of *creativity* or *ingenuity*, which people do not expect a simple computer program to have. We admire Wiles’ proof of Fermat’s last theorem; the scientific theories of Newton, Einstein, and Darwin; the design of the Golden Gate bridge and the pyramids; and sometimes even Hercule Poirot’s and Miss Marple’s analysis of a murder, precisely because they seem to require a leap that cannot be made by everyone, let alone by a simple mechanical device. I tend to disagree with this particular intuition. Note that these are all *specific* discoveries, namely, solving general problems on specific (highly important) instances. I see no reason that computers cannot do the same, as, after all, human brains (and all of nature) simply run efficient algorithms, like computers (nothing in our understanding of nature so far contradicts this observation, despite numerous speculations, writings, and beliefs to the contrary). So when we finally understand the algorithmic processes of the brain, we may indeed be able to automate the discovery of these specific achievements and perhaps many others. Indeed, the strides recently made on many frontiers of artificial intelligence suggest that computers will eventually outdo humans on almost every task.

But the question is whether we (humans or computers) can automate them *all*. Is it possible that *every* task for which verification is easy, finding a solution is not much harder? If $\mathcal{P} = \mathcal{NP}$, the answer is positive, and creativity (of this universally abundant, verifiable kind) can be completely automated, on *every* instance. Most computer scientists (including myself) believe that this is not the case for the following, more mundane empirical reason. Attempts of probably millions of person-hours across industry and academia have been invested in proving that $\mathcal{P} = \mathcal{NP}$. This monumental effort was mostly inadvertent, made up of numerous independent projects (motivated mostly by potential profit of various applications, but also by mathematical curiosity) to find efficient algorithms for *specific* optimization problems. As will be explained in Section 3.8, if *any* of them had succeeded, a proof that $\mathcal{P} = \mathcal{NP}$ would follow. However, they all failed. Is this sufficient evidence? Hard to say, but here is the current widely held belief:

Conjecture 3.6. $\mathcal{P} \neq \mathcal{NP}$.

To segue into discussing the world of $\mathcal{P} \neq \mathcal{NP}$, I mention an important *negative* consequence of the $\mathcal{P} = \mathcal{NP}$ world, which perhaps makes it a bit less utopic than it seems. In this world, every code can be broken, practically disabling all Internet security and e-commerce applications as we know them. Indeed, it was the possibility that $\mathcal{P} \neq \mathcal{NP}$ that gave birth to *complexity-based* cryptography with its numerous applications used daily by all. Somehow, the existence of (specific, structured) hard problems whose solutions can be easily checked enables the creation of unbreakable codes between parties who have never before met (as is required, e.g., for online shopping) and many other seemingly impossible tasks.

It is quite striking that hard problems, which can’t be solved, actually have applications! And so this world of $\mathcal{P} \neq \mathcal{NP}$, in which we probably live, has advantages as well. The nature of hardness required is discussed in Section 4.5, with its full utility expositied in the discussion of cryptography in Chapter 18. The intimate connection of hardness and randomness is discussed in Section 7.2.

Given this discussion, one may wonder why it is so hard to prove that indeed, $\mathcal{P} \neq \mathcal{NP}$ —it may seem completely obvious that search is much harder than verification. We shall discuss attempts to do so and difficulties encountered in Chapter 5. Before that, in this chapter, we develop a methodology of reductions and completeness that enables us to identify the *hardest* problems in $\mathcal{NP}$, which might as well be the targets of any hardness proof. These developments and understanding, enlightening and important as they are, still seem to leave us far from the resolution of $\mathcal{P}$ vs. $\mathcal{NP}$.

While I have argued here for the huge importance of this question, its resolution (in either direction: $\mathcal{P} \neq \mathcal{NP}$ or $\mathcal{P} = \mathcal{NP}$) will only be the beginning, not the end of the story. As discussed, these classes are rather coarse, and are only two of many interesting ones. If we develop techniques to resolve $\mathcal{P}$ vs. $\mathcal{NP}$, one would hope they could be sharpened to determine far more precisely the computational resources needed to invest in solving specific problems!

Much discussion and other perspectives of the $\mathcal{P}$ vs. $\mathcal{NP}$ question, its meaning and importance, appear in all computational complexity texts and surveys referenced above, as well as in the newly published survey of Aaronson [Aar16b].

We will have much more to discuss about this question, but first we take a detour and turn to discuss a related question with a strong connection to mathematics: the $\mathcal{NP}$ vs. $\text{co}\mathcal{NP}$ question.

3.5 The class $\text{co}\mathcal{NP}$, the $\mathcal{NP}$ vs. $\text{co}\mathcal{NP}$ question, and efficiently characterizable structures

We have discussed efficient computation and efficient verification. Now let us turn to define and discuss efficient *characterization* of properties. Note that attempts, mainly in combinatorics, graph theory, and optimization, to find “good” characterizations (some successful ones, as for perfect matchings and Euler tours in graphs, and some failed ones, as for Hamiltonian cycles and colorings in graphs), were central to elucidating the definitions and importance of the concepts and classes in this chapter. Many of these, and the focus on formally defining the notion of “good” (in characterizations as well as in algorithms), go back Edmonds’ early optimization papers, mainly [Edm66].

Fix a property $C \subseteq \mathbf{I}$. We already have the interpretations

- $C \in \mathcal{P}$ if it is easy to compute if an object x has property C ,
 - $C \in \mathcal{NP}$ if it is easy to certify that an object x has property C ,
- to which we now add
- $C \in \text{co}\mathcal{NP}$ if it is easy to certify that an object x *does not have* property C ,

where we formally define the class as follows.

Definition 3.7 (The class $\text{co}\mathcal{NP}$). A set C is in the class $\text{co}\mathcal{NP}$ iff its complement $\bar{C} = \mathbf{I} \setminus C$ is in $\mathcal{NP}$.

For example, the set *PRIMES* of all prime numbers is in $\text{co}\mathcal{NP}$, since its complement *COMPOSITES* is in $\mathcal{NP}$. Similarly, the set of non-Hamiltonian graphs is in $\text{co}\mathcal{NP}$, since its complement, the set of all Hamiltonian graphs, is in $\mathcal{NP}$.

While the definition of the class $\mathcal{P}$ is symmetric,¹³ the definition of the class $\mathcal{NP}$ is *asymmetric*. Having nice certificates that a given object has property C by no means automatically entails nice certificates that a given object *does not* have this property.

Indeed, when we can do both, namely, having nice certificates for both the set and its complement, we are achieving one of mathematics’ holy grails of understanding structure, namely, *necessary and sufficient* conditions, sometimes phrased as a *characterization* or a *duality theorem*. As we well know, such characterizations are rare. When insisting (as I shall) that the certificates are furthermore *short, efficiently verifiable* ones,¹⁴ such characterizations are even rarer. This leads to the following conjecture.

¹³Having a fast algorithm to determine whether an object has a property C is equivalent to having a fast algorithm for the complementary set $\bar{C}$. In other words, $\mathcal{P} = \text{co}\mathcal{P}$.

¹⁴There are many famous duality theorems in mathematics that do not conform to this strict efficiency criterion (e.g., Hilbert’s Nullstellensatz).

Conjecture 3.8. $\mathcal{NP} \neq \text{co}\mathcal{NP}$.

Note that this conjecture implies $\mathcal{P} \neq \mathcal{NP}$. We shall discuss at length refinements of this conjecture in Chapter 6 on proof complexity.

Despite the shortage of such *efficient* characterizations (namely, properties that are simultaneously in $\mathcal{NP} \cap \text{co}\mathcal{NP}$), they nontrivially exist. This class was introduced by Edmonds [Edm66], who called them problems with *good* characterization. Here is a list of some exemplary ones, following important theorems of (respectively) Menger, Dilworth, Farkas, von Neumann, and Pratt. I informally explain the $\mathcal{NP}$ and $\text{co}\mathcal{NP}$ witnesses for most, which can be seen to be efficiently verifiable. Of course, the crux is that for each of these problems, *every* instance of the problem possesses one such witness: having the property or violating it.

Efficient duality theorems: problems in $\mathcal{NP} \cap \text{co}\mathcal{NP}$

- **Graph k -connectivity.** The set of graphs in which *every* pair of vertices is connected by (a given number) k disjoint paths. Here the $\mathcal{NP}$ -witness is a collection of such k paths between every pair, and the $\text{co}\mathcal{NP}$ -witness is a *cut* of $k - 1$ vertices whose removal disconnects some pair in the graph.
- **Partial order width.** Finite partially ordered set (poset) whose largest *antichain* (a set of pairwise incomparable elements) has at least (a given number) w elements. Here the $\mathcal{NP}$ -witness is an antichain of w elements, and the $\text{co}\mathcal{NP}$ -witness is a partition of the given poset to $w - 1$ *chains* (totally ordered sets).
- **Linear programming.** Systems of consistent linear inequalities. Here an $\mathcal{NP}$ -witness is a point satisfying all inequalities. A $\text{co}\mathcal{NP}$ -witness is a linear combination of the inequalities producing the contradiction $0 > 1$.¹⁵
- **Zero-sum games.**¹⁶ Finite zero-sum games (described by a real payoff matrix) in which the first player can gain at least v (some given value). Here the $\mathcal{NP}$ -witness is a strategy for the first player (namely, a probability distribution on the rows), which guarantees her a payoff of v , and the $\text{co}\mathcal{NP}$ -witness is a strategy for the second player (namely, probability distribution on the columns), which guarantees that he pays less than v .
- **Primes.** Prime numbers. Here the $\text{co}\mathcal{NP}$ -witness is simple: two nontrivial factors of the input. The reader is encouraged to attempt to find the $\mathcal{NP}$ -witness: a short certificate of primality. It requires only very elementary number theory.¹⁷

The known relations of $\mathcal{P}$, $\mathcal{NP}$, and $\text{co}\mathcal{NP}$ are depicted in Figure 6. The examples above, of problems in $\mathcal{NP} \cap \text{co}\mathcal{NP}$ were chosen to make a point. At the time of their discovery, interest was seemingly focused only on characterizing these structures; it is not clear whether efficient algorithms for these problems were sought as well. However, with time, all these problems turned out to be in $\mathcal{P}$, and their resolutions entered the Hall of Fame of efficient algorithms. Famous examples are the ellipsoid method of Khachian [Kha79] and the interior-point method of Karmarkar [Kar84], both for Linear Programming, and the breakthrough algorithm of Agrawal, Kayal, and Saxena [AKS04] for Primes.¹⁸

Is there a moral to this story? Only that sometimes, when we have an efficient characterization of structure, we can hope for more: efficient algorithms. And conversely, a natural stepping stone toward an elusive efficient algorithm may first be to get an efficient characterization.

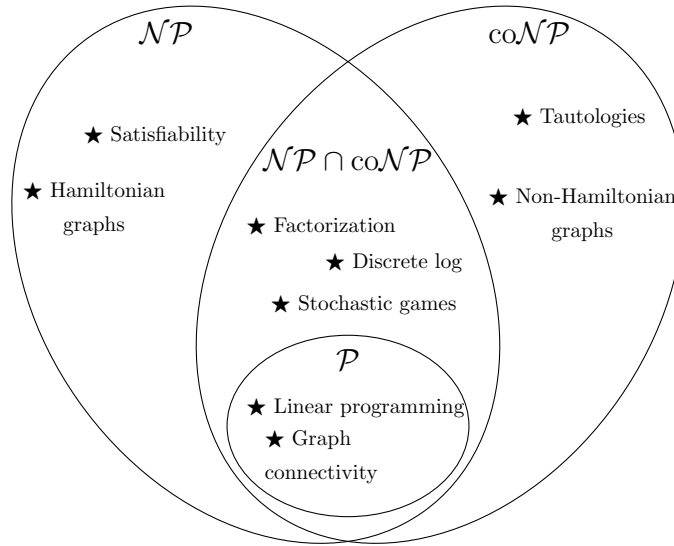
Can we expect this magic to always happen? Is $\mathcal{NP} \cap \text{co}\mathcal{NP} = \mathcal{P}$? We do not have too many examples of problems in $\mathcal{NP} \cap \text{co}\mathcal{NP}$ that have resisted efficient algorithms. Some of the famous, like integer factoring

¹⁵This duality generalizes to other convex bodies given by more general constraints, like *semi-definite* programming. Such extensions include the Kuhn-Tucker conditions and the Hahn-Banach theorem.

¹⁶This problem was later discovered to be equivalent to linear programming.

¹⁷Hint: Roughly, the witness consists of a generator of $\mathbb{Z}_p^*$, a factorization of $p - 1$, and a recursive certificate of the same type for each of the factors.

¹⁸It is interesting that assuming the Generalized Riemann Hypothesis, a simple polynomial-time algorithm was given 30 years earlier by Miller [Mil76].


 Figure 6. $\mathcal{P}$, $\mathcal{NP}$, and $\text{co}\mathcal{NP}$.

and discrete logarithms,¹⁹ arise from *one-way* functions which underlie cryptography (we discuss these in Section 4.5). Note that while they are not known to be hard, humanity literally banks on their intractability for electronic commerce security. Yet another famous example, for which membership in $\mathcal{NP}$ and in $\text{co}\mathcal{NP}$ are highly nontrivial (respectively proved in [Lac15] and [HLP99]) is the *unknottedness* problem, namely, testing whether a knot diagram represents the trivial knot. A very different example is Shapley’s *stochastic games*, studied by Condon in [Con92], for which no efficient algorithm is known. However, we have seen that many problems first proved to be in $\mathcal{NP} \cap \text{co}\mathcal{NP}$ eventually were found to be in $\mathcal{P}$. It is hard to generalize from so few examples, but the general belief is that the two classes are different.

Conjecture 3.9. $\mathcal{NP} \cap \text{co}\mathcal{NP} \neq \mathcal{P}$.

Note that this conjecture implies $\mathcal{P} \neq \mathcal{NP}$, and is implied by conjecture 3.8.

We now return to developing the main mechanism, which will help us study such questions: *efficient reductions* and *completeness*.

3.6 Reductions: A partial order of computational difficulty

In this section, we deal with relating the computational difficulty of problems for which we have no efficient solutions (yet).

Recall that we can regard any classification problem (on finitely described objects) as a subset of our set of inputs $\mathbf{I}$. Efficient reductions provide a natural partial order on such problems that captures their relative difficulty. Note that reductions are a primary tool in computability and recursion theory, from which computational complexity developed. There, reductions were typically simply *computable* functions, whereas the focus of computational complexity is on *efficiently computable* ones. While we concentrate here on time efficiency, the field studies a great variety of other resources; limiting these in reductions is as fruitful as limiting them in algorithms. The following crucial definition is depicted in Figure 7.

¹⁹Which have to be properly defined as decision problems.

Definition 3.10 (Efficient reductions). Let $C, D \in \mathbf{I}$ be two classification problems. $f: \mathbf{I} \rightarrow \mathbf{I}$ is an efficient reduction from C to D if $f \in \mathcal{P}$ and for every $x \in \mathbf{I}$ we have $x \in C$ iff $f(x) \in D$. In this case, we call f an *efficient reduction* from C to D . We write $C \leq D$ if there is an efficient reduction from C to D .

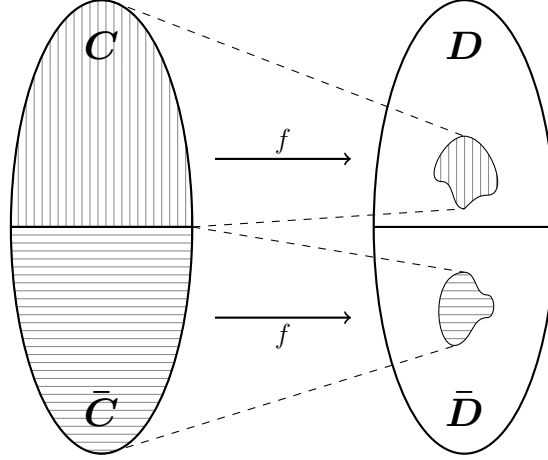


Figure 7. A schematic illustration of a reduction between two classification problems.

So, if $C \leq D$, then solving the classification problem C is computationally not much harder than solving D (up to polynomial factors in the running time).²⁰ If we have both $C \leq D$ and $D \leq C$ then C and D are computationally equivalent (again, up to polynomial factors). This gives formal meaning to the word “equivalent” in Theorem 3.1.

The definition of efficient computation (more precisely, that the composition of two polynomials is a polynomial) allows two immediate but important observations on the usefulness of efficient reductions. Please verify both for yourself! First, that indeed $\leq$ is *transitive*, and thus defines a partial order on classification problems. Second, one can compose (as in Figure 8) an efficient algorithm for one problem and an efficient reduction from a second problem to get an efficient algorithm for the second. Specifically, if $C \leq D$ and $D \in \mathcal{P}$, then also $C \in \mathcal{P}$.

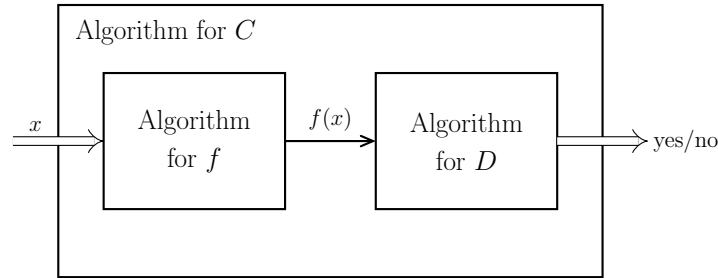


Figure 8. Composing a reduction and an algorithm to create a new algorithm.

As noted, $C \leq D$ means that solving the classification problem C is computationally not much harder than solving D . In some cases, one can replace “computationally” by the (vague) term “mathematically.” To be useful mathematically and allow better understanding of one problem in terms of another may require more properties of the reduction f than merely being efficiently computable. For example, we may want f to be a linear transformation, or a low-degree polynomial map, and indeed in some cases (as we will see e.g.

²⁰In particular, if $C \in \mathcal{P}$ then it is not much harder than trivial problems D (e.g. D might ask to distinguish sequences starting with 0 from those starting with 1). The reduction in this case would simply solve C .

in Chapter 12) this is possible. When such a connection between two classification problems (which look unrelated) can be proved, it can mean the importability of techniques from one area to another.

The power of efficient reductions to relate seemingly unrelated notions will unfold in later sections. We shall see that they can relate not only classification problems but also such diverse concepts as hardness to randomness; average-case to worst-case difficulty; proof length to computation time; and last but not least, the security of electronic transactions to the difficulty of factoring integers. In a sense, *efficient reductions are the backbone of computational complexity*. Indeed, given that polynomial time reductions can do all these wonders, no wonder we have a hard time characterizing the class $\mathcal{P}$!

3.7 Completeness: Problems capturing complexity classes

We now return to classification problems. The partial order of their difficulty, provided by efficient reductions, allows us to define the *hardest* problems in a given class of problems. Let $\mathcal{C}$ be any collection of classification problems (namely, every element of $\mathcal{C}$ is a subset C of $\mathbf{I}$). In this chapter we are mainly concerned about the class $\mathcal{C} = \mathcal{NP}$. But later in the book we will see this important idea recur for other *complexity classes* (namely classes of problems defined by the resources required to solve them, like $\mathcal{NP}$).

Definition 3.11 (Hardness and completeness). A problem D is called $\mathcal{C}$ -hard if for every $C \in \mathcal{C}$, we have $C \leq D$. If we further have that if $D \in \mathcal{C}$, then D is called $\mathcal{C}$ -complete.

In other words, if D is $\mathcal{C}$ -complete, it is a hardest problem in the class $\mathcal{C}$: if we manage to solve D efficiently, we have done so automatically for all other problems in $\mathcal{C}$.

It is not a priori clear that a given class of has any complete problems! It is perhaps a miracle of computational complexity that many natural *complexity classes* do! Note that a given class may have many complete problems, and by definition, they all have essentially the same complexity. If we manage to prove that *any* of them cannot be efficiently solved, then we automatically have done so for *all* of them.

It is trivial (and uninteresting) that every problem in the class $\mathcal{P}$ is in fact $\mathcal{P}$ -complete under our definition. It becomes interesting when we find such universal problems in classes of problems for which we do not have efficient algorithms. By far, the most important of all such classes is $\mathcal{NP}$.

3.8 $\mathcal{NP}$ -completeness

As mentioned earlier, the seminal papers of Cook [Coo71] and Levin [Lev73] defined $\mathcal{NP}$, efficient reducibilities and completeness, but their crowning achievement was the discovery of a *natural* $\mathcal{NP}$ -complete problem.

Definition 3.12 (The problem *SAT*). A Boolean formula is a logical expression over Boolean variables (that can take values in $\{0, 1\}$) with connectives $\wedge, \vee, \neg$ (standing for AND, OR, NOT), for example, $(x_1 \vee x_2) \wedge (\neg x_3)$. Let *SAT* denote the set of all *satisfiable* Boolean formulas (namely, those for which a Boolean assignment to the variables evaluates to 1).

For example, the following formula is unsatisfiable:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3) \wedge (x_1 \vee x_3 \vee x_4) \wedge (\neg x_4 \vee x_1),$$

while the one below is satisfiable:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_1 \vee x_3 \vee x_4) \wedge (\neg x_4 \vee x_1).$$

We now arrive at a foundational theorem of computational complexity, revealing the primary importance of this simple-looking problem of satisfying Boolean formulas.

Theorem 3.13 [Coo71, Lev73]. *SAT is $\mathcal{NP}$ -complete.*

Recall the meaning of that statement. For *every* set $C \in \mathcal{NP}$, there is an efficient reduction $f: \mathbf{I} \rightarrow \mathbf{I}$ with the following property: for every sequence $z \in \mathbf{I}$, we have that $z \in C$ iff the sequence $f(z)$ encodes a satisfiable formula.

The proof of this theorem (namely, the construction of the reduction algorithm f) gives an extra bonus that turns out to be extremely useful: It maps witnesses to witnesses.²¹ More precisely, for any witness y certifying that $z \in C$ (via some verifier V_C), the same reduction f converts the witness y to a Boolean assignment of the variables of the formula $f(z)$ that satisfy it. In other words, this reduction translates not only between the *decision* problems, but also between the associated *search* problems.

A few words are in order about the proof of Theorem 3.13. It is of course easy to see that SAT is in $\mathcal{NP}$; a satisfying assignment is an easy-to-verify witness. The difficult part is proving the $\mathcal{NP}$ -hardness of SAT . Certainly the proof cannot afford to consider every problem $C \in \mathcal{NP}$ separately. The gist of the proof is a *generic* transformation: taking a description of the verifier V_C for C and emulating its computation on input z and hypothetical witness y to efficiently create a Boolean formula $f(z)$ (whose variables are the bits of y). The formula constructed simply tests the validity of the computation of V_C on (z, y) and checks that this computation outputs 1. Here the locality and simplicity of the individual steps of algorithms (say, described as Turing machines) play a central role: Checking the consistency of each step of the computation of V_C amounts essentially to a constant size formula on a few bits.

To summarize, SAT captures the difficulty of the whole class $\mathcal{NP}$. In particular, the $\mathcal{P}$ vs. $\mathcal{NP}$ problem can now be phrased as a question about the complexity of *one* problem instead of infinitely many of them.

Corollary 3.14. $\mathcal{P} = \mathcal{NP}$ iff $SAT \in \mathcal{P}$.

A great advantage of having one complete problem at hand (like SAT) is that now, to prove that another problem (say, $D \in \mathcal{NP}$) is $\mathcal{NP}$ -complete, we only need to design a reduction from SAT to D (namely, prove $SAT \leq D$). We already know that for *every* $C \in \mathcal{NP}$, we have $C \leq SAT$, and transitivity of $\leq$ takes care of the rest.

This idea was powerfully used in the next seminal paper, that of Karp [Kar72]. In his paper, Karp listed 21 problems from logic, graph theory, scheduling, and geometry, and showed them to be $\mathcal{NP}$ -complete. This was the first demonstration of the wide spectrum of $\mathcal{NP}$ -complete problems, and it initiated an industry of finding more such problems. A few years later, Garey and Johnson [GJ79] published their book on $\mathcal{NP}$ -completeness, which contains hundreds of such problems from diverse branches of science, engineering, and mathematics. Today, thousands of these problems are known. We will soon discuss the meaning and importance of this notion, but first I give some examples of $\mathcal{NP}$ -complete problems, and the nature of the connection among them.

3.9 Some $\mathcal{NP}$ -complete problems

We stress again that all $\mathcal{NP}$ -complete problems are equivalent in a very strong sense. Any algorithm solving one can be simply translated into an equally efficient²² algorithm solving any other.

We are finally ready to see the proof of Theorem 3.1 on the equivalence of our motivating examples from Section 3.1. It follows from the following three theorems.

Theorem 3.15 [AM75]. *The set 2DIO is $\mathcal{NP}$ -complete.*

Theorem 3.16 [AHT06]. *The set KNOT is $\mathcal{NP}$ -complete.*

Theorem 3.17 [Kar72, Sto73]. *The set 3COL is $\mathcal{NP}$ -complete.*

Recall that to prove $\mathcal{NP}$ -completeness of a set, one has to prove two things: that it is in $\mathcal{NP}$, and that it is $\mathcal{NP}$ -hard. In almost all $\mathcal{NP}$ -complete problems, membership in $\mathcal{NP}$ (namely, the existence of short certificates) is easy to prove. Certainly, a candidate 3-coloring of a given map is short and easy to check. For 2DIO, one can easily see that if there is a positive integer solution (x, y) to $Ax^2 + By + C = 0$, then in fact there is short one,²³ indeed a solution whose length (in bits) is linear in the lengths of A, B, C . Thus, a short witness is simply a root (x, y) . But KNOT is an exception, and the short witnesses for the knot having a small genus requires Haken's algorithmic theory of normal surfaces, considerably enhanced

²¹We will see one crucial use of this property in the proof of Theorem 10.5 on *zero-knowledge proofs*.

²²As usual, up to polynomial factors.

²³Hint: If (x, y) is a root, so is $(x + B, y - A(2x + B))$.

(even short certificates for unknottedness in $\mathbb{R}^3$ are hard to obtain; see [HLP99]). Let us discuss what these $\mathcal{NP}$ -completeness results mean, first about the relationship between the three sets, and then about each individually.

The proofs that these problems are complete follow by reductions from (variants of) *SAT*. The discrete, combinatorial nature of these reductions may cast doubt on the possibility that the computational equivalence of these problems implies the ability of real “technology transfer” between, for example, topology and number theory. Nevertheless, now that we know of the equivalence, perhaps simpler and more direct reductions can be found between these problems. Moreover, we stress again that reductions translate between witnesses as well. Namely, for any instance, say $(M, K, G) \in \text{KNOT}$, if we translate it using this reduction to an instance $(A, B, C) \in \text{2DIO}$ and happen (either by sheer luck or special structure of that equation) to find an integer root, the same reduction will translate that root back to a description of a genus G manifold that bounds the knot K . Today many such $\mathcal{NP}$ -complete problems are known throughout mathematics, and for some pairs, the equivalence can be mathematically meaningful and useful (as it is between some pairs, of computational problems).

Let us discuss the simplest of the three reductions above, namely, from *SAT* to *3COL*. If you have never seen one, it should be a mystery: The two problems talk about different worlds, one of logic and the other of graph theory. Both are difficult problems, but the reduction should be easy (namely, efficiently computable). The key to this reduction, as well as to almost any other, is the locality of computation! This of course is evident in *SAT*; a formula is composed from Boolean gates, each of which performs a simple, local operation. However, *3COL* feels like a more global property.²⁴ The idea of this reduction is to focus on the individual gates of the input formula. We’ll find a reduction that works for each gate and will compose the small (“gadget”) graphs produced, mimicking the structure prescribed by the input formula. Let’s elaborate this idea.

Here is how to transform the satisfiability problem for the (trivial, 1-gate) formula $x \vee y$ to a graph 3-coloring problem. We will actually transform the equation $x \vee y = z$ to a graph 3-coloring statement using the gadget graph shown in Figure 9. Check that it satisfies the following condition: In *every* legal 3-coloring of the graph with colors $\{0, 1, 2\}$, the colors of the vertices labeled x, y, z will be from $\{0, 1\}$, which will satisfy the equation $x \vee y = z$. One can easily construct such gadgets for the gates $\wedge, \neg$ as well. Now, to complete the reduction, the algorithm proceeds as follows. Given an arbitrary formula as input, it names its wires, builds a gadget graph for every gate, and identifies appropriate vertices in these to generate an output graph. By construction, it is 3-colorable if and only if the given formula was satisfiable. This is essentially the reduction in [Kar72], but we are not done yet: The gadget graph above is not planar (and hence the output graphs are also not planar). However, Stockmeyer [Sto73] gives another gadget that can eliminate crossings in planar embeddings of graphs without changing their 3-colorability. The reader is encouraged to find such a gadget. With this, the proof of Theorem 3.17 is complete.

We now list a few more $\mathcal{NP}$ -complete problems of a different nature, to give a feeling for the breadth of this phenomenon. Some appear already in Karp’s original article [Kar72]. Again, hundreds more can be found in Garey and Johnson’s book, [GJ79], and by now, many thousands are known:

- **Hamiltonian cycle.** Given a graph, is there a simple cycle of edges going through every vertex precisely once?
- **Subset-sum.** Given a sequence of integers $a_1, \dots, a_n$ and b , is there a subset J such that $\sum_{i \in J} a_i = b$?
- **Integer programming.** Given a polytope in $\mathbb{R}^n$ (by its bounding hyperplanes), does it contain an integer point?
- **Clique.** Given a graph and an integer k , are there k vertices with all pairs mutually adjacent?
- **Quadratic equations.** Given a system of multivariate polynomial equations of degree at most 2, over a finite field (say, $\mathbb{F}_2$), do they have a common root?

²⁴Consider, for example, a cycle on n vertices, where n is odd; it requires 3 colors, but if we remove any edge, it can be 2-colored.

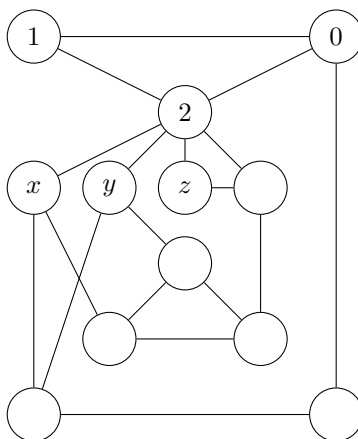


Figure 9. The gadget underlying the reduction from *SAT* to *3COL*.

- **Shortest lattice vector.** Given a lattice L in $\mathbb{R}^n$ and an integer k , is the shortest nonzero vector of L of (Euclidean) length $\leq k$?

3.10 The nature and impact of $\mathcal{NP}$ -completeness

$\mathcal{NP}$ -completeness is a unique scientific discovery—there seems to be no precisely defined scientific notion that even comes close to being pervasive in so many fields of science and engineering! We start with its most immediate impact, in computer science itself, move to mathematics, and then to science and beyond. Some of this discussion will become more meaningful (and impressive) as you read further through the book, and in detail in the last chapter. More can be found in, for example, Papadimitriou’s retrospective on the subject [Pap97]. Curiously, that paper reports that electronic search (new at the time) revealed thousands of science and math papers with the phrase $\mathcal{NP}$ -complete in them; today, more than 20 years later, this number is in the millions!

As mentioned, starting with Karp’s paper [Kar72], an explosion of $\mathcal{NP}$ -completeness results followed quickly in every corner and subfield of computer science. This is easy to explain. Most individuals in the field of computer science, from academics to industry programmers, are busy seeking efficient algorithms for numerous computational problems. How can one justify failure to find such an algorithm? In the absence of any techniques for proving intractability, the next best thing was proving that the computational problem at hand was $\mathcal{NP}$ -complete (or $\mathcal{NP}$ -hard), which means that finding such an efficient algorithm for it would imply an efficient algorithm for numerous others, which many others failed to solve. In short, failing to prove $\mathcal{P} = \mathcal{NP}$ is a very powerful excuse, and $\mathcal{NP}$ -completeness is an excellent stamp of hardness. Every professional of the field knows this! While $\mathcal{NP}$ -completeness is a negative result (basically showing that what we want is impossible), such negative results had a positive impact. As problems do not go away when you declare them $\mathcal{NP}$ -complete and still demand solutions, weaker solution concepts for them were developed. For example, for optimization problems, people attempted to find good approximation algorithms. Moreover, given that $\mathcal{NP}$ -completeness only captures worst-case hardness, people developed algorithms that work well “on average” and heuristics that seem to work well on inputs that “show up in practice.” This direction needs lots more theory, especially in light of the recent practical success of *deep networks*, that is far from understood. A variety of quality criteria and models were developed for different relaxations of efficient solvability, leading to analogous complexity theories, which enable researchers to argue hardness as well; some of these models will be discussed later in the book. A major such theory, able to argue $\mathcal{NP}$ -completeness for approximation problems, and actually pinpoint in many cases the exact limits of efficient approximation, will be discussed in Section 10.4.

The next field to be impacted by $\mathcal{NP}$ -completeness was mathematics. With some delay, $\mathcal{NP}$ -completeness

theorems started showing up in most mathematical disciplines, including algebra, analysis, geometry, topology, combinatorics, and number theory. This “intrusion” may seem surprising, as most questions that mathematicians ask themselves are not algorithmic. However, existence theorems for a variety of objects beg the question of having “explicit” descriptions of such objects. Moreover, in many fields, one actually needs to find such objects. Mathematics is full of a variety of constructions, done by hand long ago and by numerous libraries of computer programs that are essential for progress, and hence, their efficiency is also essential. Like computer scientists, mathematicians adopted the notion of polynomial-time algorithm as a first cut at defining “efficient” and “explicit.” Thus, description and construction problems that are $\mathcal{NP}$ -complete were extremely useful to set limits on the hopes of achieving these properties. Furthermore, in mathematics, such $\mathcal{NP}$ -completeness results signified an underlying “mathematical nastiness” of the structures under study. For example, as explained in Section 3.5, an efficient characterization of a property that is $\mathcal{NP}$ -complete will imply that $\mathcal{NP} = \text{co}\mathcal{NP}$, and so it is unlikely (as we understand things today) that such a characterization exists. As in CS, so in math as well, such bad news begets good outcomes, nudging mathematicians into more productive directions: refining or specializing the properties under study, considering a variety of approximate notions, or simply being satisfied with sufficient and necessary conditions that are not complementary (as is needed for characterization).

The presence and impact on $\mathcal{NP}$ -completeness in science is evidenced by the fact that such results (which are patently about computation) in biology, chemistry, economics, neuroscience, electrical engineering, and other fields are being proved not by computer scientists but by biologists, chemists, economists, neuroscientists, electrical engineers, and so forth. Moreover, these results are being published in the scientific journals of these very fields. And the numbers are staggering: A search for papers that contain the phrase “ $\mathcal{NP}$ -complete” or “ $\mathcal{NP}$ -completeness” prominently (in the title, abstract, or keywords) reveals that in *each* of these disciplines, there are *hundreds* of such papers, and many thousands more that mention these terms in the body of the paper. To obtain such results, these thousands of scientists needed to learn the concepts and proof methods of computational complexity, typically a foreign language to most (for instance, mathematical theorems rarely appear in science articles at all).

This phenomenon begs an explanation! Indeed, there are two questions to answer. What explains the abundance of $\mathcal{NP}$ -completeness in these diverse disciplines? And why do their scientists bother making the unusual effort to prove these computational theorems?

One important observation is that scientists often study processes and try to build models that explain and predict them. Almost by definition, these are computational processes, namely, composed of a sequence of *simple, local steps*, like Turing machines, albeit manipulating not bits in computers but possibly neurons in the brain, proteins in the cell, atoms in matter, fish in a school, or stars in a galaxy. In other words, many models simply describe *algorithms* that nature uses for generating certain processes or behavior. A typical $\mathcal{NP}$ -completeness result often refers to the limits of prediction by a particular model of some natural process. Here are some illustrative examples. In some existing models, it is $\mathcal{NP}$ -complete to compute the following quantities: the minimal surface area that a given foam will settle into (in physics), the minimal energy configuration of a certain molecule (e.g., as in protein folding in biology), and the maximum social welfare of certain equilibria (in economics). Let’s explore the meaning of such results to modeling natural phenomena.

We’ll make two natural assumptions, which seem completely benign. First, that $\mathcal{P} \neq \mathcal{NP}$. Then $\mathcal{NP}$ -completeness means that no efficient algorithm can compute the required quantities (e.g., in the examples just mentioned), at least for *some* instances. Second, that natural processes are inherently efficient algorithms, and so are the measurements we perform to extract these quantities at the end of the process. These two assumptions clash with each other, which seems to suggest at least one of two possible conclusions. One possibility is that the model (for which $\mathcal{NP}$ -completeness was proved) is simply wrong (or incomplete) in describing reality. The other possibility is that the “hard” instances simply never occur in nature.²⁵ In both cases, $\mathcal{NP}$ -completeness calls for a better understanding (e.g., refinement of the model at hand), a characterization of the instances for which the algorithm suggested by the model solves efficiently (and an

²⁵For example, it is quite possible that over billions of years of evolution, only proteins that are easily and efficiently foldable survived, and others became extinct.

argument that these conditions are consistent with what we see in nature).

This idea has caused some researchers to propose that our underlying conjecture, $\mathcal{P} \neq \mathcal{NP}$, should be viewed as a *law of nature*. Perhaps the first explicit such occurrence is this quote,²⁶ from Volker Strassen’s laudation [Str86] for Les Valiant on his Nevanlinna Prize: “*The evidence in favor of Cook’s and Valiant’s hypotheses is so overwhelming, and the consequences of their failure are so grotesque, that their status may perhaps be compared to that of physical laws rather than that of ordinary mathematical conjectures.*” Note that at that time, the utility of this mathematical conjecture to science was not as well understood as it is today. How to precisely articulate this mathematical statement as a relevant law of nature is of course still interesting to debate. An intuitive desire is to let it play the same role that the second law of thermodynamics plays in science: Scientists would be extremely wary to propose a model that violates it. One suggestion, by Scott Aaronson, is a stronger statement about the real world: *There are no physical means to solve $\mathcal{NP}$ -complete problems in polynomial time.* A host of possible physical means that were actually attempted is discussed in [Aar05].

There is still the mystery of the ubiquitous presence of $\mathcal{NP}$ -completeness in practically every subfield of CS, math, and essentially all sciences. With hindsight, this is an amplified (and much more relevant) incarnation of the ubiquity of undecidability in all these disciplines. Both are explained by the fact that computation, viewed (as above) as any process evolving via a sequence of simple, local steps, is so ubiquitous. Similarly, descriptions of properties of systems with many parts (either desired properties or observed properties, which are typically the outcomes of such computations) are often given or modeled by sets of simple, local constraints on small subsystems of the whole. As it happens, for almost all choices of constraints, their mutual satisfaction for given instances is undecidable if the system is infinite and is $\mathcal{NP}$ -complete if finite. In much rarer cases, they lead to (respectively) decidable or polynomial-time solvable problems. Understanding these phenomena, and delineating the types of constraints across the tractable/intractable barrier, is an active field of study and will be discussed further in Section 4.3.

Concluding this somewhat philosophical section, we note another major impact of $\mathcal{NP}$ -completeness. Namely, that it served as a role model for numerous other notions of computational universality. $\mathcal{NP}$ -completeness turned out to be an extremely flexible and extendible notion, allowing numerous variants, which enabled capturing universality in other (mainly computational, but not only) contexts. It led to the definitions of classes of problems solvable using very different resource bounds. In most cases, these classes were also shown to have complete problems, capturing the difficulty of the whole class under natural reductions, with the benefits described above (some examples will be discussed in Section 4.1 and then in later chapters). Much of the whole evolution of computational complexity, the theory of algorithms, and most other areas in theoretical computer science has been guided by the powerful approach of reduction and completeness. This progression has generated multiple theories of *intractability* in various settings and tools to understand and possibly curb or circumvent it (even though we are still mostly unable to prove intractability in most cases). The structures revealed by this powerful methodology send an important message to other disciplines.

I feel that the strongest impact of this message will be on the sciences. I will elaborate on this idea in Chapter 20 and summarize it here in one paragraph. The discussion in this chapter suggests integrating computational complexity aspects into *every* model of natural processes. In other words, scientists will come to account not only for the mechanism by which things evolve, and the way in which physical quantities are affected, but also for the amounts of relevant computational resources expended in that evolution. Doing so will add constraints that may guide scientists to better models, which nature can actually “carry out.” When taking computational constraints into account in scientific models becomes standard practice, the way science is done will be revolutionized. While such a paradigm shift will take time to propagate across science, it is in the works! The abovementioned occurrence of $\mathcal{NP}$ -completeness in so many scientific papers is just the beginning and illustrates both the importance of complexity to science and the willingness of scientists to embrace it. But beyond this, in more and more works (indeed, primarily collaborative works of computer scientists with scientists in other fields), we see models that integrate computational aspects into

²⁶In this quote Cook’s hypothesis is $\mathcal{P} \neq \mathcal{NP}$, and Valiant’s hypothesis is what became known as $\mathcal{VP} \neq \mathcal{VNP}$ which we will discuss in Chapter 12.

the description of natural processes and how this fusion can lead to radically new scientific insights. A few of the books, surveys, and articles representing this trend in various disciplines include [NRTV07, EK10, Val00, Kar11, HH13, Val13, CLPV14, Pap14], among many others. By way of example, [HH13], by physicists Daniel Harlow and Patrick Hayden, offers a complexity-theoretic based explanation that is possibly the only way out of the famous “black hole firewall paradox.” Much more on these exciting developments can be found in Chapter 20.